

Part I — GRPO Finetuning of gemma-3-1b-it on GSM8K

Multi-Agent Systems and Agentic AI — Practical Examination

Team **chmawa** (sm3035, bc654, zw499) · written individually by Zixuan Wang (zw499)

Jointly owned (shared team work): the v6e-1 TPU, the six shared GRPO runs R0–R5, and the code repository.

All prose, figure selection and interpretation below are my own.

Code: github.com/wdk0082/a8-grpo (to be ported to GitLab for submission) | W&B: [a8-grpo](#) | [GRPO](#), [DAPO](#), [LoRA](#), [GSM8K](#)

I.1 Reproducing the baseline

(i) **The run.** We reproduce the baseline as run **R0** ([8c2785ut](#)): upstream commit [borisbolliet/tpu-2026@77c5a67](#) with the `config.py` defaults ($\beta=0.08$, $G=2$), full **3364** GRPO steps (one epoch), **~5 h 05 m** wall-clock on **v6e-1**.

(ii) **Accuracy.** Held-out GSM8K *test*, HF split, fixed seed 42, greedy/exact-match. We report both the `config.py` default $n=64$ and the full $n=1319$ (for completeness; I.3 standardises on the full test) as ($n=64/n=1319$): base **45.3 / 47.4%**; R0 best-val **42.2 / 46.7%** (step 700) and final **0.0 / 6.2%** (step 3364). At either n the baseline *over-optimises* — R0 stays within noise of base then collapses by the final step (the format reward saturates while correctness decays); the wide $n=64$ CIs (± 12 pt) are why I.3 uses the full test (per-checkpoint CIs in Table 1).

(iii) **Training curves.** Fig. 1 (R0, black): mean reward $\bar{r}$ rises, peaks $\approx$ step 1000, then declines to ~ 0.5 ; $\text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$ climbs from 0 (an early spike to ~ 4 , settling ~ 1.6) — the over-optimisation arc.

Baseline patches (as-shipped fixes; full diff: `tpu-2026_our_changes.patch` in repo). (1) `run_tmux.sh` hard-coded a foreign home path; (2) the W&B entity defaulted to a third party — redirected to `a8-grpo`; (3) `evaluate.py` built a fresh $B=0$ LoRA and never restored a checkpoint, so it could only score *base* — we added Orbax `-step/-ckpt-dir` LoRA restore; (4) TFDS’s `gsm8k` builder crashes under `protobuf 7.x` once JAX imports, so we evaluate on the identical GSM8K test via an HF `datasets` fallback (seed 42). (5) Raised checkpoint retention (`SAVE_INTERVAL_STEPS` 500 $\rightarrow$ 100, `MAX_TO_KEEP` 4 $\rightarrow$ 40) so the best-val checkpoint survives for selection — a persistence change, not a training one.

I.2 Understanding the baseline

(a) **Policies & trainable parameters.** π_θ is the LoRA-adapted policy: frozen `gemma-3-1b-it` weights plus low-rank adapters $\Delta W=BA$; **only A, B are trainable** (the base is frozen). π_{old} is the policy that generated the current rollouts — with `NUM_ITERATIONS` $\mu=1$ there is one gradient step per rollout batch, so $\pi_{\text{old}}=\pi_\theta$ at the update. π_{ref} is the frozen initial policy for the KL penalty; LoRA initialises $B=0$, so $\pi_\theta=\pi_{\text{ref}}$ exactly at step 0.

(b) **Group size & estimator.** $K=\text{NUM_GENERATIONS}=2$. Tunix uses `advantage_estimator='grpo'`: the **standard** group-mean, std-normalised advantage $\hat{A}_i=(r_i - \bar{r})/\sigma_r$ with $\bar{r}, \sigma_r$ over all K samples *including* i — *not* leave-one-out.

(c) **Shaping vs. correctness rewards.** Shaping: `match_format_exactly` and `match_format_approximately` (reward the `<reasoning>/<answer>` template). True correctness: `check_answer` and `check_numbers` (extracted number vs. gold). With the correctness reward *alone*, an early policy is almost never numerically right, so every sample in a group scores ≈ 0 : within-group $\sigma_r \rightarrow 0$ and $\hat{A}_i=(r_i - \bar{r})/\sigma_r$ has no signal (an ε in the denominator averts a literal 0/0) — degenerate groups, no gradient. The shaping terms give differently-formatted samples different rewards, restoring $\sigma_r > 0$ and a usable advantage that bootstraps learning (the same $\sigma_r \rightarrow 0$ degeneracy analysed in I.4 Q1).

(d) **Clipped surrogate & ϵ .** The PPO-style clip lives in Tunix’s `GRPOLearner` (module `tunix.rl`), in its clipped policy-gradient loss: it forms a **per-token** ratio $\rho_t=\pi_\theta(a_t)/\pi_{\text{old}}(a_t)$ and optimises $\min(\rho_t \hat{A}, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \hat{A})$, with $\epsilon=\text{EPSILON}=0.2$. Two deviations from the lecture form: (i) the ratio is **per-token**, not per-sequence; (ii) at $\mu=1$, $\pi_{\text{old}}=\pi_\theta \Rightarrow \rho_t \equiv 1$, so the clip is **dormant** in the baseline (it engages only for $\mu > 1$).

Table 1: Held-out accuracy on the *full* GSM8K test ($n=1319$, HF split, seed 42), greedy/exact-match, $\pm 95\%$ bootstrap CI. Last column: *paired* bootstrap Δ vs. base over the shared prompts (`paired_ci.py`). * = paired 95% CI excludes 0; † = collapse. R1($G2, \beta.3$) and R2($G2, +len$) fell *below* base (R1 best 37.2 / final 0.0; R2 best 32.8 / final 40.6) — clean negative results.

Model	(G, β), step	Acc. (%)	95% CI	Δ base (paired)
Base <code>gemma-3-1b-it</code>	—	47.4	44.7–50.0	—
R0 (baseline)	(2, .08) best 700	46.7	44.0–49.4	−0.7 n.s.
R0	(2, .08) final 3364	6.2	4.9–7.5	−41.2†
R4	(2, 0) best 1300	52.8	50.1–55.5	+5.4*
R4	(2, 0) final 3364	48.6	45.9–51.3	+1.2 n.s.
R3b	(8, 0) best 1000	55.7	53.0–58.2	+8.3*
R3b	(8, 0) final 3364	57.2	54.6–59.9	+9.9*
R5	(8, .08) best 1300	53.4	50.7–56.2	+6.1*
R5	(8, .08) final 3364	55.6	52.8–58.2	+8.2*

I.3 Improving on the baseline

Modification & motivation. The baseline over-optimises at $K=2$. The theory of I.4 Q1 gives the GRPO gradient variance $\text{Var}(\hat{g}) = K^{-2} \sum_i (h_i - \bar{h})(h_i - \bar{h})^\top = O(1/K)$, with effective sample size $K_{\text{eff}} \propto K$: a larger group directly lowers the variance of the gradient estimate $\hat{g}$. We therefore take **group size G** (the K of I.2(b) and the theory) as the primary lever and run a controlled $\beta \times G$ **2×2** (KL-penalty weight $\times$ group size) to separate it from the KL leash: R4($G2, \beta0$), R0($G2, \beta.08$; the baseline), R3b($G8, \beta0$), R5($G8, \beta.08$) — plus single-axis controls R1($\beta=0.3$) and R2(+soft length penalty), for **six full runs**. All share data (HF GSM8K, seed 42), eval, horizon (3364 steps) and a seed-matched rollout RNG (`PRNGKey(0)`); only the named knob changes per run.

We report **(a)** held-out accuracy (Table 1), **(b)** mean reward and KL for all runs on shared axes (Fig. 1), and **(c)** a diagnostic for a known GRPO failure mode — group degeneracy at $G=8$ (Fig. 2b). **(d) Findings vs. theory.** $G=8$ is the winning intervention. Both $G=8$ runs beat base — R3b +9.9, R5 +8.2pp, paired CIs excluding 0 — and each beats its matched $G=2$ run (R3b–R4 = +8.6pp at final; R5–R0 = +49pp). **R3b ($G=8, \beta=0$) is the best model at 57.2%.** The 2×2 (Fig. 2a) localises the failure: *the collapse is confined to the $G=2, \beta>0$ corner* (R0 final 6.2%, R1 0%), while the *same $\beta=0.08$ at $G=8$ (R5) is stable*, and at $G=8$ the KL weight is statistically irrelevant (R3b vs. R5: +1.7pp, n.s.). Furthermore *over-optimisation is itself a $G=2$ phenomenon*: $G=2$ peaks at best-val then degrades (R4 52.8→48.6; R0 collapses), whereas $G=8$ **improves monotonically to the final step** (final > best). This is precisely the I.4 Q1 mechanism: larger K shrinks $\text{Var}(\hat{g})$, so updates are smoother — the policy neither reward-hacks into R0’s format-only collapse nor (once $\beta>0$) trips the $\exp(\log\text{-ratio})$ KL-estimator instability behind R1’s blow-up.

Contradicted prediction. We predicted an over-optimising baseline would need a *tighter* KL leash. The opposite held: raising β to 0.3 (R1) collapsed *faster* (0%), and removing the KL term entirely (R4, $\beta=0$) was fine — the cure was variance reduction (group size), not regularisation, echoing DAPO’s removal of the KL term for long-output RL. The diagnostic (Fig. 2b) shows the cost of $G=8$: $\approx \frac{1}{3}$ of groups are degenerate (all- K -equal $\Rightarrow$ zero advantage), the K_{eff} /dead-group issue of I.4 Q1 and the target of DAPO’s Dynamic Sampling — yet the net effect of more rollouts is still +8–10pp. R1 and R2 fell below base (Table 1), honest negative results that further support variance, not extra shaping or a stronger leash, as the operative lever.

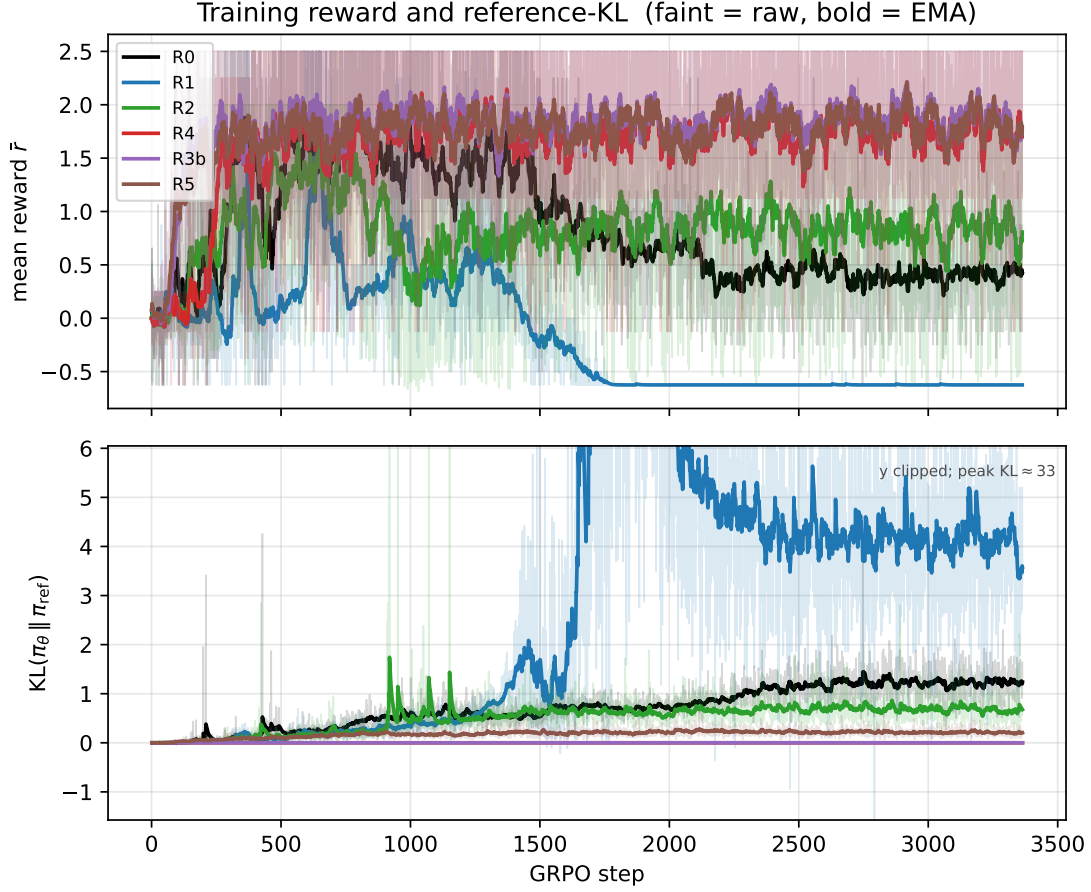


Figure 1: Mean training reward $\bar{r}$ (top) and $KL(\pi_\theta \parallel \pi_{ref})$ (bottom) vs. GRPO step, for all six runs on shared axes. R0 over-optimises (reward peaks then declines) and R1 collapses (KL blow-up ~step 1700); the $G=8$ runs (R3b, R5) and R4 stay stable.

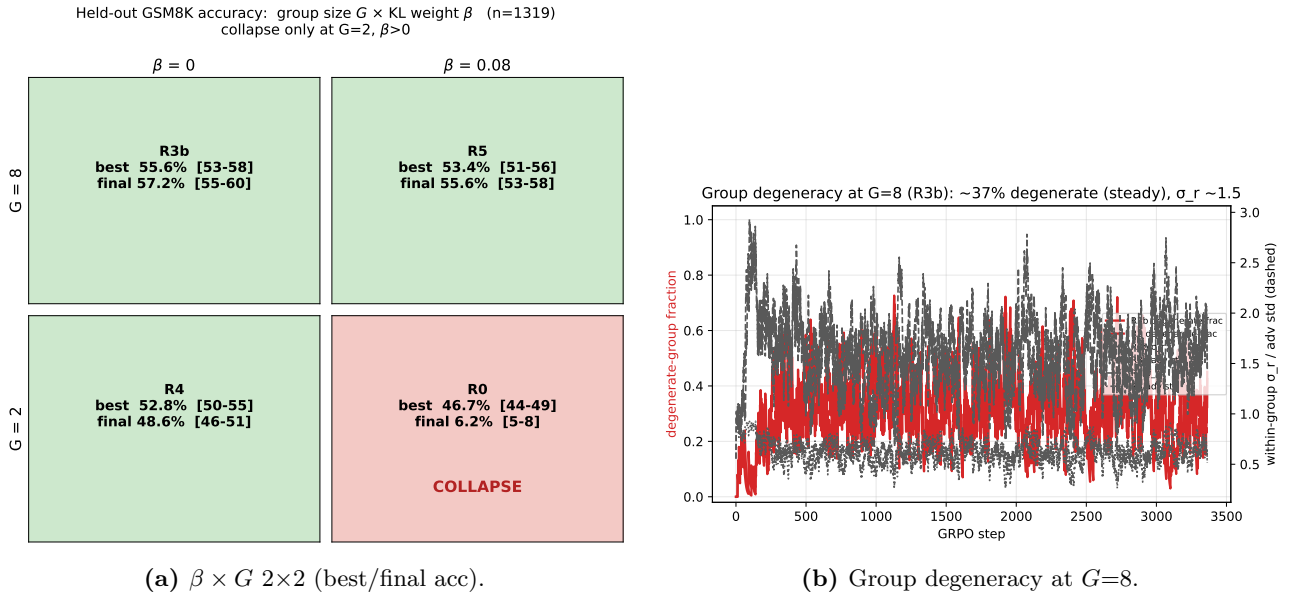


Figure 2: Left: accuracy by (G, β) — collapse occurs *only* at $G=2, \beta>0$ (R0). Right: the diagnostic — at $G=8$ about a third of groups are degenerate (all- K rewards equal) yet σ_r holds ≈ 1.5 (steady, not $\rightarrow 0$).

I.4 GRPO theory — written questions (40 marks)

I.4.1 Group-mean normalisation

Let

$$\bar{h} = \frac{1}{K} \sum_{i=1}^K h_i.$$

(i) Since $\mathbb{E}[r_i] = \mathbb{E}[\bar{r}] = \mu$,

$$\mathbb{E}[\hat{A}_i] = \frac{\mathbb{E}[r_i - \bar{r}]}{\sigma} = 0.$$

Therefore

$$\boxed{\mathbb{E}[X_i] = 0}, \quad \boxed{\mathbb{E}[\hat{g}] = 0}.$$

Here the responses are conditioned on and treated as fixed, while the artificial reward noise is independent of them. In the usual policy-gradient problem the responses are sampled from the policy and their rewards depend on the sampled responses, so the policy gradient need not vanish.

(ii) The common random term is the sample mean $\bar{r}$, appearing in every

$$X_i = \frac{h_i}{\sigma} (r_i - \bar{r}).$$

For $i \neq j$,

$$\begin{aligned} \text{Cov}(r_i - \bar{r}, r_j - \bar{r}) &= \text{Cov}(r_i, r_j) - \text{Cov}(r_i, \bar{r}) - \text{Cov}(\bar{r}, r_j) + \text{Var}(\bar{r}) \\ &= 0 - \frac{\sigma^2}{K} - \frac{\sigma^2}{K} + \frac{\sigma^2}{K} \\ &= -\frac{\sigma^2}{K}. \end{aligned}$$

Hence

$$\boxed{\text{Cov}(X_i, X_j) = -\frac{1}{K} h_i h_j^\top, \quad i \neq j.}$$

More precisely, the scalar covariance of the centred rewards is negative; the cross-covariance matrix is a negative scalar multiple of $h_i h_j^\top$. The negative dependence follows from the exact constraint

$$\sum_{i=1}^K (r_i - \bar{r}) = 0 :$$

if one centred reward increases, the others must collectively decrease.

(iii) The full covariance matrix is

$$\boxed{\text{Cov}(\hat{g}) = \frac{1}{K^2} \left[\sum_{i=1}^K h_i h_i^\top - \frac{1}{K} \left(\sum_{i=1}^K h_i \right) \left(\sum_{j=1}^K h_j \right)^\top \right] = \frac{1}{K^2} \sum_{i=1}^K (h_i - \bar{h})(h_i - \bar{h})^\top.}$$

For the scalar variance convention used in the lectures, define

$$\text{Var}_{\text{tr}}(Z) := \text{tr}(\text{Cov}(Z)).$$

Taking the trace gives

$$\boxed{\text{Var}_{\text{tr}}(\hat{g}) = \frac{1}{K^2} \sum_{i=1}^K \|h_i - \bar{h}\|^2.}$$

Also,

$$\text{Cov}(X_1) = \left(1 - \frac{1}{K}\right) h_1 h_1^\top,$$

and hence

$$\text{Var}_{\text{tr}}(X_1) = \left(1 - \frac{1}{K}\right) \|h_1\|^2.$$

If one instead averaged K independent copies of X_1 , the naive i.i.d. variance would be

$$\text{Var}_{\text{tr,iid}}(\hat{g}) = \frac{1}{K} \text{Var}_{\text{tr}}(X_1).$$

Following the lecture convention, define the effective sample size by

$$\text{Var}_{\text{tr}}(\hat{g}) = \frac{\text{Var}_{\text{tr}}(X_1)}{K_{\text{eff}}}.$$

Therefore

$$K_{\text{eff}} = \frac{\text{Var}_{\text{tr}}(X_1)}{\text{Var}_{\text{tr}}(\hat{g})} = \frac{K^2 \left(1 - \frac{1}{K}\right) \|h_1\|^2}{\sum_{i=1}^K \|h_i - \bar{h}\|^2} = \frac{K(K-1) \|h_1\|^2}{\sum_{i=1}^K \|h_i - \bar{h}\|^2}.$$

Equivalently, its discrepancy from the nominal sample size K is

$$\frac{K_{\text{eff}}}{K} = \frac{\text{Var}_{\text{tr,iid}}(\hat{g})}{\text{Var}_{\text{tr}}(\hat{g})}.$$

If

$$\frac{1}{K} \sum_{i=1}^K \|h_i - \bar{h}\|^2$$

remains of constant order and $\|h_1\|^2 = O(1)$, then

$$K_{\text{eff}} = \Theta(K)$$

as $K \rightarrow \infty$.

If all score vectors lie in one direction, write

$$h_i = c_i v.$$

Then

$$\text{Cov}(\hat{g}) = \frac{v v^\top}{K^2} \sum_{i=1}^K (c_i - \bar{c})^2,$$

so the covariance has rank at most one, and

$$K_{\text{eff}} = \frac{K(K-1) c_1^2}{\sum_{i=1}^K (c_i - \bar{c})^2}.$$

Thus only variation in the scalar coefficients c_i contributes to the variance. In the special case $h_i = h$ for every i ,

$$\hat{g} = \frac{h}{K} \sum_{i=1}^K \hat{A}_i = 0$$

for every reward realisation. Hence $\text{Cov}(\hat{g}) = 0$ and, provided $h \neq 0$, the effective sample size is formally infinite.

I.4.2 KL-regularised policy update

(i) The objective is

$$F(\pi) = \sum_a \pi(a) A_t(a) - \frac{1}{\eta} \sum_a \pi(a) \log \frac{\pi(a)}{\pi_t(a)}.$$

Introducing a Lagrange multiplier λ for $\sum_a \pi(a) = 1$, stationarity on the support of π_t gives

$$A_t(a) - \frac{1}{\eta} \left(\log \frac{\pi(a)}{\pi_t(a)} + 1 \right) + \lambda = 0.$$

Consequently,

$$\pi_{t+1}(a) = \frac{\pi_t(a) \exp(\eta A_t(a))}{\sum_b \pi_t(b) \exp(\eta A_t(b))}.$$

Actions outside the support of π_t remain assigned zero probability. The objective is strictly concave on this support, so the optimiser is unique, assuming the normalising constant is finite.

For small η ,

$$\pi_{t+1}(a) = \pi_t(a) [1 + \eta (A_t(a) - \mathbb{E}_{\pi_t}[A_t])] + O(\eta^2),$$

so η controls the update size. Large η places increasingly more mass on high-advantage actions.

(ii) For the true advantage of π_t ,

$$J(\pi_t) = \mathbb{E}_{a \sim \pi_t}[A^{\pi_t}(a)] = 0.$$

The regularised objective evaluated at π_t is also zero. Since π_{t+1} maximises it,

$$J(\pi_{t+1}) - \frac{1}{\eta} \text{KL}(\pi_{t+1} \| \pi_t) \geq 0.$$

It follows that

$$J(\pi_{t+1}) \geq \frac{1}{\eta} \text{KL}(\pi_{t+1} \| \pi_t) \geq 0 = J(\pi_t).$$

Thus the update guarantees monotone improvement in the scalar performance measure defined in the question.

In neural-network GRPO the guarantee may fail because the exact advantage is replaced by a noisy finite-sample estimate and the maximisation is replaced by a finite number of stochastic-gradient steps in a restricted parametric policy class. The clipped GRPO objective is also not identical to the exact KL-regularised objective above.

(iii) For one sample, write

$$\ell(\rho, \hat{A}) = \min \left(\rho \hat{A}, \text{clip}(\rho, 1 - \epsilon, 1 + \epsilon) \hat{A} \right).$$

Equivalently,

$$\ell(\rho, \hat{A}) = \begin{cases} \hat{A} \min(\rho, 1 + \epsilon), & \hat{A} \geq 0, \\ \hat{A} \max(\rho, 1 - \epsilon), & \hat{A} < 0. \end{cases}$$

Thus, for a positive advantage, the objective gives no further incentive to increase the ratio above $1 + \epsilon$. For a negative advantage, it gives no further incentive to reduce the ratio below $1 - \epsilon$.

This is not a hard constraint

$$\rho \in [1 - \epsilon, 1 + \epsilon].$$

Other samples can still move the ratio outside this interval. In contrast, a constraint

$$\text{KL}(\pi_\theta(\cdot | q) \| \pi_{\text{old}}(\cdot | q)) \leq \delta$$

is a joint distribution-level constraint coupling all actions. PPO clipping is samplewise and advantage-dependent, does not directly constrain unsampled actions, and does not guarantee that the joint KL divergence is small.

I.4.3 Mixture proposal and importance weighting

Write

$$p(a) = \pi_{\text{old}}(a \mid q), \quad u(a) = \pi_{\text{unif}}(a \mid q), \quad m(a) = (1 - \alpha)p(a) + \alpha u(a),$$

and let

$$s_\theta(a) = \nabla_\theta \log \pi_\theta(a \mid q).$$

For the population analysis, define

$$V_p = \mathbb{E}_{a \sim p}[R(q, a)], \quad \sigma_p^2 = \text{Var}_{a \sim p}[R(q, a)], \quad A_p(a) = \frac{R(q, a) - V_p}{\sigma_p}.$$

(i) The target gradient is

$$g_p = \mathbb{E}_{a \sim p}[s_\theta(a) A_p(a)].$$

Since $m(a) \geq (1 - \alpha)p(a)$, the proposal has support wherever p does. The importance weight is

$$w(a) = \frac{p(a)}{m(a)} = \frac{\pi_{\text{old}}(a \mid q)}{(1 - \alpha)\pi_{\text{old}}(a \mid q) + \alpha\pi_{\text{unif}}(a \mid q)}.$$

Therefore

$$g_p = \mathbb{E}_{a \sim m}[w(a)s_\theta(a)A_p(a)],$$

and an importance-corrected estimator is

$$\hat{g}_{\text{mix}} = \frac{1}{K} \sum_{i=1}^K w_i s_\theta(a_i) \hat{A}_i^{(p)}, \quad w_i = w(a_i),$$

where $\hat{A}_i^{(p)}$ must estimate the old-policy advantage rather than the mixture-policy advantage.

(ii) As $\alpha \rightarrow 0$,

$$m(a) \rightarrow p(a), \quad w(a) \rightarrow 1.$$

Moreover, the reweighted old-policy moment estimates given below reduce to the ordinary empirical moments. Hence, for the same realised actions,

$$\hat{g}_{\text{mix}} \rightarrow \hat{g}_{\text{GRPO}}.$$

(iii-a) By linearity of expectation,

$$V^{\pi_{\text{mix}}}(q) = (1 - \alpha)V^{\pi_{\text{old}}}(q) + \alpha V^{\pi_{\text{unif}}}(q).$$

(iii-b) At the population level, for any deterministic baseline b ,

$$\mathbb{E}_m[w(a)s_\theta(a)(R(q, a) - b)] = \mathbb{E}_p[s_\theta(a)(R(q, a) - b)].$$

In particular,

$$\begin{aligned} \mathbb{E}_m[ws_\theta(R - V^{\pi_{\text{mix}}})] &= \mathbb{E}_p[s_\theta(R - V^{\pi_{\text{old}}})] \\ &\quad + (V^{\pi_{\text{old}}} - V^{\pi_{\text{mix}}}) \mathbb{E}_p[s_\theta]. \end{aligned}$$

At the policy-gradient evaluation point $\pi_\theta = p$,

$$\mathbb{E}_p[s_\theta] = \mathbb{E}_p[\nabla_\theta \log p(a)] = 0,$$

so a deterministic baseline shift alone does not bias the gradient.

However, the empirical $\bar{r}$ is a random function of the same group and the empirical σ_r is also proposal-dependent. Multiplying only the outer gradient contribution by w_i therefore does not transform mixture group normalisation into old-policy group normalisation.

The intended empirical correction is to use weighted moments

$$S_w = \sum_{j=1}^K w_j, \quad \bar{r}_w = \frac{\sum_{j=1}^K w_j r_j}{S_w},$$

and

$$(\sigma_{r,w})^2 = \frac{\sum_{j=1}^K w_j (r_j - \bar{r}_w)^2}{S_w}.$$

The corresponding reweighted advantage is

$$\boxed{\hat{A}_i^{(w)} = \frac{r_i - \bar{r}_w}{\sigma_{r,w}},}$$

giving

$$\boxed{\hat{g}_{\text{mix}} = \frac{1}{K} \sum_{i=1}^K w_i s_\theta(a_i) \frac{r_i - \bar{r}_w}{\sigma_{r,w}}.}$$

This self-normalised estimator is consistent for the old-policy moments, but, like ordinary same-group GRPO normalisation, it is not exactly unbiased at finite K .

For a literally unbiased finite- K baseline correction, one may instead use the leave-one-out Horvitz–Thompson baseline

$$\hat{V}_{p,-i} = \frac{1}{K-1} \sum_{j \neq i} w_j r_j, \quad \mathbb{E}[\hat{V}_{p,-i}] = V_p.$$

Since $\hat{V}_{p,-i}$ is independent of a_i ,

$$\mathbb{E}_m \left[w_i s_\theta(a_i) \frac{r_i - \hat{V}_{p,-i}}{\sigma_p} \right] = \mathbb{E}_p \left[s_\theta(a) \frac{R(q, a) - V_p}{\sigma_p} \right].$$

An estimated standard deviation must similarly be obtained independently if literal finite-sample unbiasedness is required.

(iv) First ignore the additional randomness from estimating the baseline and standard deviation. Let

$$Y(a) = s_\theta(a) A_p(a), \quad g_p = \mathbb{E}_p[Y(a)].$$

Then

$$\text{Var}(\hat{g}_{\text{GRPO}}) = \frac{1}{K} \left(\mathbb{E}_p[YY^\top] - g_p g_p^\top \right),$$

whereas

$$\text{Var}(\hat{g}_{\text{mix}}) = \frac{1}{K} \left(\mathbb{E}_m[w^2 YY^\top] - g_p g_p^\top \right) = \frac{1}{K} \left(\mathbb{E}_p[w YY^\top] - g_p g_p^\top \right).$$

Therefore

$$\boxed{\text{Var}(\hat{g}_{\text{mix}}) - \text{Var}(\hat{g}_{\text{GRPO}}) = \frac{1}{K} \mathbb{E}_p \left[(w - 1) YY^\top \right].}$$

There is no universal ordering between the two covariance matrices. In a direction v ,

$$v^\top [\text{Var}(\hat{g}_{\text{mix}}) - \text{Var}(\hat{g}_{\text{GRPO}})] v = \frac{1}{K} \mathbb{E}_p \left[(w - 1) (v^\top Y)^2 \right].$$

Moreover,

$$w(a) - 1 = \frac{\alpha(p(a) - u(a))}{m(a)}.$$

Thus mixing increases variance when large squared gradient contributions, and hence large squared reward deviations, are concentrated on responses for which $p(a) > u(a)$. Those responses are under-sampled by the mixture relative to p and receive weights $w(a) > 1$.

Conversely, mixing decreases variance when large reward deviations are concentrated on responses for which $u(a) > p(a)$. The uniform component oversamples these responses and their weights satisfy $w(a) < 1$.

The change in the unweighted reward variance can also be seen directly:

$$\text{Var}_m(R) = (1 - \alpha) \text{Var}_p(R) + \alpha \text{Var}_u(R) + \alpha(1 - \alpha)(V_p - V_u)^2.$$

Hence mixture sampling increases the raw reward variance if

$$\text{Var}_u(R) + (1 - \alpha)(V_p - V_u)^2 > \text{Var}_p(R),$$

and decreases it if the reverse inequality holds. Importance weighting adds the separate w^2 effect described above.

(v-a) Under the stated assumption,

$$w_i = c^T = \exp(T \log c).$$

Since $0 < c < 1$, $\log c < 0$, and therefore

$$w_i \rightarrow 0 \quad \text{exponentially fast as } T \rightarrow \infty.$$

A ratio below one is typical for exploratory tokens or responses that are unlikely under π_{old} but receive additional probability from the uniform component, so that $\pi_{\text{mix}} > \pi_{\text{old}}$. It is not implied for every token merely by $\alpha > 0$.

(v-b) Such trajectories make exponentially small contributions to the estimator. If many sampled trajectories have collapsed weights, the estimate is effectively based on only a small number of old-policy-like trajectories, causing a small effective sample size and unstable estimates; the products may also underflow numerically.

One unbiased remedy is to keep the proposal closer to the old policy, for example by reducing α or enforcing the per-token log-ratio condition

$$\left| \log \frac{\pi_{\text{old}}(a_t \mid q, a_{<t})}{\pi_{\text{mix}}(a_t \mid q, a_{<t})} \right| \leq \frac{\delta}{T}.$$

Then

$$e^{-\delta} \leq w_i \leq e^{\delta},$$

so the full-response weight cannot collapse exponentially. The trade-off is weaker uniform exploration. Alternatively, clipping or self-normalising the response weights reduces variance and numerical collapse, but introduces bias.